

Chapter 3. 에이전트는 왜, 어떻게 실패하는가

이 챕터에서 배우는 것 (이런 분이 먼저 읽으면 좋다: "LLM이 가끔 이상한 답을 한다"는 걸 알지만, 그게 구체적으로 어떤 패턴으로 나타나고 어떻게 잡아낼 수 있는지 실제 사례로 확인하고 싶은 분)

- Book-forge 개발 중 실제로 관측된 5가지 실패 유형
- 각 실패가 왜 일어나는지 — 모델의 한계인지, 코드의 한계인지
- 이 실패들이 3부·4부의 Harness Engineering 메커니즘과 어떻게 정확히 하나씩 연결되는지

이 챕터의 사례는 전부 이 저장소를 실제로 개발하며 실측으로 관측된 것이다 — 가정이 아니라 로그와 커밋에 남은 기록이다.

3.1 무응답 — 모델이 아무 답도 안 낸다

1장에서 이미 언급한 사례다. 추론(thinking) 모델(`qwen3.6:35b-mlx` 등)은 "생각"을 먼저 하고 나서 답을 낸다. `num_predict` (토큰 예산)를 그 생각 과정이 다 써버리면, 최종 응답이 빈 문자열로 끝난다. `provider.py` 가 이를 실측으로 확인하고 `think: False` 를 강제해 막았다. 이 실패의 특징은 **에러가 나지 않는다**는 것이다. HTTP 요청은 200 OK로 성공하고, `response` 필드만 비어 있다. 겉보기엔 아무 문제 없이 파이프라인이 끝까지 돌아가지만, 결과 파일은 텅 비어 있다.

3.2 환각 — 존재하지 않는 것을 사실처럼 말한다

이 책을 준비하는 과정에서 직접 재현됐다. Book-forge/Agent-Evaluator 소스를 근거로 Book-forge 자신에 대한 책을 자동 생성해봤더니, 생성된 챕터 하나가

`BranchGuard` 라는 클래스를 언급했다. 그럴듯하게 들리지만, 실제 클래스 이름은 `BranchGuardConfig` (`agent_evaluator/gates/branch_guard.py`)다. LLM은 문맥상 "브랜치를 지키는 무언가"가 있어야 한다는 것까지는 정확히 추론했지만, 정확한 이름은 지어낸 셈이다.

더 미묘한 사례도 있다. `code_consistency_checker.py`의 주석에 남아 있는 실제 버그 기록이다.

"실측 확인: `Settings` (`agent_evaluator.config`),
`KoreanRAGDatasetGenerator` (`agent_evaluator.datasets`),
`LiveGuardrail` (`agent_evaluator.gates.live_guardrail`)는 전부 실존하는 클래스인데 최상위 재노출이 없어 `_resolve_dotted`가 매번 오탐(없음)으로 판정했다."

이 경우는 **LLM이 아니라 검증 도구 자신의 한계**였다. 클래스는 실제로 존재하는데, 최상위 패키지 네임스페이스에 재노출(re-export)돼 있지 않아서 검증기가 "찾을 수 없다"고 잘못 판정한 것이다. 이 사례는 중요한 교훈을 남긴다. **검증 도구도 틀릴 수 있고, 그 도구 자체를 검증하는 두 번째 확인이 필요하다**(11장에서 이 버그가 어떻게 고쳐졌는지 다룬다).

3.3 드리프트 — 서로 다른 챕터가 같은 내용으로 수렴한다

배치로 여러 챕터를 한 번에 재생성(`--all`)할 때 관측된 패턴이다. 서로 무관한 주제였던 두 챕터가 같은 소스 청크를 반복해서 검색 결과 상위에 뽑으면, 둘 다 그 내용으로 쓸려버릴 수 있다. 실제로 서로 다른 주제였던 두 챕터가 둘 다 같은 모듈(예: 특정 토큰나이저 관련 내용)로 드리프트한 사례가 있었다. 원인은 RAG 검색 자체의 성질에 있다. 특정 소스 파일의 청크가 유난히 임베딩 공간에서 "중심적인" 위치에 있으면, 서로 다른 질의(챕터 제목)에도 반복해서 상위로 뽑힌다. `knowledge/store.py`의 `max_per_source` (실제 코드는 7장 § 7.3에서 확인한다)가 정확히 이 문제, 즉 "한 소스가 검색 결과를 독점하는 것"을 막기 위해 추가됐다.

3.4 반복 — 같은 행동을 무한히 되풀이한다

`scaffold.py`가 챕터 스텝 파일을 만들 때, 이론적으로는 같은 도구 호출이 계속 반복될 위험이 있다(예: 재시도 로직에 버그가 있어 같은 파일 쓰기를 계속 시도하는 경우).

`scaffold_project()`는 이 위험에 대비해 아예 세션 전체를 감싼다.

 파일: `src/book_forge/agents/scaffold.py`

```
guardrail = LiveGuardrail(loop_detection=LoopDetectionConfig(consecutive_repeat_threshold=5))
with live_guardrail_session(guardrail, task_id=f"scaffold_{project_dir.name}"):
    for spec in chapters:
        results.append(write_chapter_stub(...))
```

`consecutive_repeat_threshold=5`는 같은 호출이 5번 연속 반복되면 차단한다는 뜻이다. 이 메커니즘은 14장에서 훨씬 자세하게 다룬다. 여기서 강조할 것은, 이 방어가 **"모델이 반복할 수도 있다"**는 것을 전제로 미리 짜여 있다는 점이다. 실제로 반복이 관측된 뒤에 추가한 사후 대응이 아니라, LLM 기반 시스템이라면 구조적으로 있을 수 있는 위험을 처음부터 가정한 설계다.

3.5 경로 침범 — 허용된 범위 밖에 쓴다

`write_chapter_stub()` (`scaffold.py`)는 함수 시작부에서 곧바로 경로를 검사한다.

 파일: `src/book_forge/agents/scaffold.py` (`write_chapter_stub()`)

```
resolved_project = project_path.resolve()
resolved_target = chapter_path.resolve()
if resolved_project not in resolved_target.parents:
    raise BookForgeError(f"프로젝트 디렉토리 밖 경로 쓰기 시도 차단: {resolved_target}")
```

이 검사가 왜 `ScopeConfig` (Agent-Evaluator SDK가 제공하는 Harness Config) 대신 직접 짠 코드인지는 이 함수 바로 위 주석이 정확히 밝힌다. `ScopeConfig`는 파일 경로가 아니라 **도구 이름**(allow/forbid 목록)을 검사하는 설정이기 때문이다. "프로젝트 디렉토리 밖에 쓰지 말라"는 요구는 SDK가 대신 해주는 기능이 아니라, 이 모듈이 직접

구현해야 하는 방어 코드였다. SDK에 있는 기능과 없는 기능을 정확히 구분하지 않으면, "Config 하나만 켜면 다 막아준다"는 잘못된 믿음으로 이어질 수 있다.

3.6 다섯 실패 유형과 이 책의 대응 지점

실패 유형	원인의 성격	이 책에서 다루는 대응
무응답	provider 구현 세부사항	1장(<code>think: False</code>)
환각	모델의 한계 + 검증 도구 자신의 한계	9장(Gate C), 11장(정적 검증기)
드리프트	RAG 검색의 구조적 성질	7장(<code>max_per_source</code>)
반복	LLM 기반 시스템의 구조적 위험	14장(<code>LoopDetectionConfig</code>)
경로 침범	부작용이 있는 도구 호출	14장(경로 검사 + <code>tool_guard</code>)

 **개발자 TIP:** 이 표를 거꾸로 읽으면 체크리스트가 된다. 새 에이전트를 만들 때 "이 다섯 유형 중 어느 것이 이 에이전트에 해당하는가"를 먼저 물어보면, 어떤 Harness Config나 검증기를 붙여야 할지 판단하기 쉬워진다.

직접 해보기

§ 3.6의 표(다섯 실패 유형 → 원인의 성격)를 체크리스트 삼아, 여러분이 만들었거나 만들 계획인 에이전트에 하나씩 대입해보자. "이 에이전트는 무응답 위험이 있는가(추론 모델을 쓰는가)?", "환각 위험이 있는가(사실을 생성하는가)?", "반복 위험이 있는가(도구를 반복 호출할 수 있는가)?", "경로 침범 위험이 있는가(파일을 쓰는가)?" 넷 다 "아니오"인 순수 텍스트 생성 함수라면 Gate A(목표 달성) 하나만으로도 충분할 수 있다. 반대로 여러 개가 "예"라면, 이 책 3~4부가 다루는 메커니즘 중 어느 것이 필요한지 이 시점부터 미리 가늠할 수 있다.

이 챕터의 핵심

- **에이전트의 실패는 무작위가 아니라 패턴이 있다.** 무응답·환각·드리프트·반복·경로 침범 다섯 가지로 분류할 수 있다.
- **검증 도구 자신도 틀릴 수 있다.**
`Settings` / `KoreanRAGDatasetGenerator` / `LiveGuardrail` 오탐 사례가 이를 실제로 보여준다.
- **Book-forge의 방어는 사후 대응이 아니라 상당수가 사전 설계다.**
`LoopDetectionConfig` 나 경로 검사는 문제가 실제로 터지기 전부터 구조적으로 있을 수 있는 위험을 가정하고 짜여 있다.

참고 자료

- `src/book_forge/llm/provider.py` — `think: False` 관련 주석
- `src/book_forge/agents/code_consistency_checker.py` — `_walk_package_symbols()` 의 오탐 수정 경위
- `src/book_forge/knowledge/store.py` — `max_per_source`
- `src/book_forge/agents/scaffold.py` — `LoopDetectionConfig`, 경로 검사

Part II는 이 실패들을 배경 삼아, Book-forge가 에이전트 하나가 아니라 **여러 에이전트**를 어떻게 **협업시켜** 더 나은 결과를 만드는지 다룬다.